Graphs — Full Revision Sheet

> **Striver A2Z Step 15** · P01–P42 · All Problems in One File

---

How to Use This File

Each entry has: **Problem** → **Recall** → **Algo** → **Don't Miss** → **Complexity** → **Pattern Trigger**

Use this for last-minute interview prep or daily revision. For full dry runs and code, go to the individual problem file.

Quick Topic Index

Topic	Problems
Foundations	P01–P06
Grid BFS/DFS	P08–P16
Graph Properties	P07, P17–P19
Topological Sort	P20–P21
Shortest Path	P22–P36
MST	P37–P40
DSU Applications	P39–P42

P01 — Graph and Types

Recall: Undirected vs Directed. Weighted vs Unweighted. Cyclic vs Acyclic. Dense (E≈V²) vs Sparse (E≈V).

Don't miss: - DAG = Directed Acyclic Graph → enables topological sort - Tree = connected undirected graph with V-1 edges, no cycle - Degree: undirected = edges per node. Directed: in-degree + out-degree

---

P02/P03 — Graph Representation

Recall: Two ways — Adjacency Matrix and Adjacency List.

Adjacency Matrix: `matrix[u][v]=weight`. Space O(V²). O(1) edge check. **Adjacency List:** `vector<pair<int,int>> adj[V]`. Space O(V+E). O(degree) edge check.

Don't miss: - Undirected: add edge both ways (`adj[u].push_back(v)` AND `adj[v].push_back(u)`) - Weighted: `adj[u].push_back({v, w})` - Use adjacency list for sparse graphs (almost always in competitive programming)

P04 — Connected Components

Recall: Run BFS/DFS from each unvisited node. Each launch = one new component.

Algo:

```
for i in 0..V:
    if !vis[i]: components++; BFS/DFS(i)
```

Don't miss: Disconnected graphs need the outer loop — DFS from node 0 alone misses other components.

Time: O(V+E) | **Space:** O(V)

P05 — BFS

Recall: Level-order traversal. Queue. All neighbors at distance k before distance k+1.

Algo: `vis[src]=1`, push src. While queue: pop → push unvisited neighbors.

Don't miss: - Mark visited on **enqueue** (not dequeue) — prevents duplicates in queue - BFS = shortest path for **unit weight** graphs

Time: O(V+E) | **Space:** O(V)

Pattern: Min steps / shortest path with unit cost → BFS

P06 — DFS

Recall: Go deep before wide. Recursive (call stack) or iterative (explicit stack).

Algo: `vis[node]=1` → recurse all unvisited neighbors.

Don't miss: - DFS doesn't give shortest path (use BFS for that) - Recursion stack depth = O(V) worst case (chain graph)

Time: O(V+E) | **Space:** O(V)

Pattern: Explore all paths / component membership / cycle detection → DFS

P07 — Number of Provinces *(LC #547)*

Recall: Count connected components in adjacency matrix.

Algo: Outer loop → if unvisited → count++ → DFS marking all reachable.

Don't miss: - Input is adjacency **matrix** → iterate columns (not adj list) - `isConnected[i][i]=1` — skip same index or it's caught by `vis` check

DSU approach: `union(i,j)` for every `isConnected[i][j]=1`. Count roots: `parent[i]==i`.

Time: O(n^2) | **Space:** O(n)

Pattern: Connected components → outer loop + DFS/BFS or DSU

P08 — Flood Fill *(LC #733)*

Recall: BFS/DFS from source pixel. Replace all connected same-color pixels with new color.

Algo: Record `initialColor`. BFS from `(sr,sc)`. Change if `mat[r][c]==initialColor`.

Don't miss: - If `image[sr][sc]==color` (already target color) → return immediately (infinite loop otherwise) - Check `initialColor`, not current color, since you modify as you go

Time: O(n×m) | **Space:** O(n×m)

Pattern: Fill connected region with new value → BFS/DFS from source with color check

---

P09 — Rotten Oranges *(LC #994)*

Recall: Multi-source BFS. All rotten oranges start simultaneously. Track time.

Algo: Push all rotten `{0,{r,c}}` to queue. BFS: spread rot to adjacent fresh. `time=max(time,t)`. Post-check: any unvisited fresh → return -1.

Don't miss: - Multi-source: push ALL initial rotten oranges before BFS starts - Check `vis[i][j]!=2 && grid[i][j]==1` after BFS for unreachable fresh oranges - Separate `vis[][]` from `grid[][]` to not corrupt input

Time: O(n×m) | **Space:** O(n×m)

Pattern: "Simultaneous spread from multiple sources" → multi-source BFS

P10 — Cycle Detection Undirected (BFS)

Recall: BFS with `{node, parent}`. If unvisited neighbor → push. If visited AND not parent → cycle.

Algo: `queue<pair<int,int>>` storing `{node, parent}`. For each neighbor: if `!vis` push; if `vis && neighbor!=parent` → cycle.

Don't miss: - Check `neighbor != parent`, not `neighbor != -1` — need to skip the edge we came from - Run for all unvisited nodes (disconnected graph)

Time: O(V+E) | **Space:** O(V)

P11 — Cycle Detection Undirected (DFS)

Recall: DFS with parent tracking. If visited neighbor is NOT parent → back edge → cycle.

Algo: `dfs(node, parent, adj, vis)`. For each neighbor: if `!vis` → recurse; if `vis && adj!=parent` → return true.

Don't miss: - Same logic as BFS but recursive - Build adj from edges before calling DFS

Time: O(V+E) | **Space:** O(V)

P12 — 0/1 Matrix *(LC #542)*

Recall: Multi-source BFS from all 0-cells simultaneously. Distances to 1-cells fill naturally.

Algo: Push all `{0,{r,c}}` (cells with value 0). BFS outward: `dist[newRow][newCol]=steps+1`. Assign on dequeue.

Don't miss: - Initialize `dist[r][c]=0` for all 0-cells, `1e9` for 1-cells before BFS - Multi-source from 0s, not 1s (find distance TO nearest 0)

Time: O(n×m) | **Space:** O(n×m)

Pattern: Distance to nearest special cell → multi-source BFS from those cells

---

P13 — Surrounded Regions *(LC #130)*

Recall: Any 'O' connected to border cannot be captured. DFS/BFS from border 'O's → mark safe → flip remaining.

Algo: DFS from all border 'O's → mark as 'T'. Then: 'O'→'X', 'T'→'O'.

Don't miss: - Start DFS from **border cells only**, not all 'O's - Three-value pass: 'O'(unsafe)→'X', 'T'(safe)→'O'

Time: O(n×m) | **Space:** O(n×m)

Pattern: "Regions connected to boundary survive" → reverse thinking, DFS from border

---

P14 — Number of Enclaves *(LC #1020)*

Recall: Count '1' cells NOT reachable from any border. BFS from all border '1's → remaining unvisited '1's are enclaves.

Algo: Push all border land cells. BFS marks reachable. Count `grid[i][j]==1 && !vis[i][j]`.

Don't miss: - Border check: `i==0 || j==0 || i==n-1 || j==m-1` - Count unvisited land (not visited land) at end

Time: O(n×m) | **Space:** O(n×m)

Pattern: Count cells that can't escape to border → BFS from border, count remainder

---

P15 — Number of Islands *(LC #200)*

Recall: Count connected components of '1' cells. DFS in-place marks visited by flipping '1'→'0'.

Algo: Outer loop. If `grid[i][j]=='1'` : count++, DFS (flip to '0'). 4-directional.

Don't miss: - In-place marking avoids separate `vis[][]` - Grid stores chars ('0','1'), not ints

Time: O(n×m) | **Space:** O(n×m) recursion stack

Pattern: Connected components on 2D grid → DFS with in-place marking

---

P16 — Number of Distinct Islands *(GFG)*

Recall: Count islands with distinct shapes. Encode shape as relative coordinates from DFS start.

Algo: DFS from each unvisited '1'. Record `{r-baseR, c-baseC}` for each cell visited. Store in `set<vector<pair<int,int>>>`.

Don't miss: - Relative coordinates (subtract start position) make shape position-independent - Two islands with same relative coords = same shape → set deduplicates

Time: O(n×m×log(n×m)) | **Space:** O(n×m)

Pattern: Distinct connected components → normalize shape via relative coordinates + set

---

P17 — Bipartite Graph *(LC #785)*

Recall: 2-colorable = bipartite. BFS/DFS assigning alternating colors. Conflict = not bipartite.

Algo: BFS: `color[adj] = !color[node]`. If `color[adj]==color[node]` → not bipartite.

Don't miss: - `!color[node]` not `1-color[node]` (same thing, but use `!` for booleans) - Run for all unvisited nodes (disconnected graph) - Odd cycle ↔ not bipartite

Time: O(V+E) | **Space:** O(V)

Pattern: Can nodes be 2-colored? → BFS coloring, check conflict

---

P18 — Cycle Detection Directed (DFS)

Recall: Two arrays: `vis[]` (ever visited) and `pathVis[]` (on current DFS path). Back edge on path → cycle.

Algo: `vis[node]=1; pathVis[node]=1`. Recurse. If neighbor `vis && pathVis` → cycle. **Backtrack:** `pathVis[node]=0` on return.

Don't miss: - `pathVis[node]=0` on backtrack — critical. Forgetting this gives false cycles. - `vis[node] && !pathVis[node]` → already processed, safe (cross edge)

Time: O(V+E) | **Space:** O(V)

Pattern: Directed cycle → DFS + vis + pathVis. Backtrack pathVis.

P19 — Eventual Safe States *(LC #802)*

Recall: A node is "safe" if all paths from it eventually reach a terminal node (no cycle). Three approaches.

Approach 1 (DFS): `vis[]`, `pathVis[]`, `check[]`. `check[node]=1` only if DFS returns false (no cycle found from this node).

Approach 2 (3-state): `state[]=0(unvisited),1(visiting),2(safe)`. `state==1` during DFS → cycle → unsafe.

Approach 3 (Kahn's): Reverse all edges. Nodes with in-degree 0 in reversed graph = safe. BFS (Kahn's) from them.

Don't miss: - `check[node]` set after ALL neighbors processed (not before) - Safe = no cycle reachable from node

Time: O(V+E) | **Space:** O(V)

P20 — Topological Sort *(GFG)*

Recall: Linear ordering of vertices where for every edge u→v, u appears before v. Only for DAGs.

DFS approach: DFS + stack. Push node to stack AFTER all neighbors processed. Reverse stack = topo order.

BFS (Kahn's): Compute in-degrees. Push all 0-in-degree nodes. BFS: for each neighbor, in-degree--; if 0 → push.

Don't miss: - Only valid for DAGs (no cycles) - Kahn's: if processed nodes < V → cycle exists

Time: O(V+E) | **Space:** O(V)

Pattern: "Process prerequisites first" → topological sort

P21 — Alien Dictionary *(GFG)*

Recall: Build character graph from adjacent word pairs. Apply Kahn's topo sort on character graph.

Algo: Compare adjacent words char by char → first mismatch gives directed edge `c1→c2`. Then topo sort on 26-node graph.

Don't miss: - Compare only ADJACENT words in the list (not all pairs) - If `word1` is prefix of `word2` but comes after it → invalid → return "" - Result of topo sort gives the alien character order

Time: O(N×L + 26) where N=words, L=avg length | **Space:** O(26)

Pattern: Infer ordering from comparisons → build directed graph + topo sort

---

P22 — Shortest Path Undirected Unit Weight *(GFG)*

Recall: BFS from source. `dist[adj]=dist[node]+1` on relaxation. Unit weight → BFS works.

Don't miss: BFS not Dijkstra — unit weights make BFS O(V+E) vs Dijkstra's O((V+E)logV).

Time: O(V+E) | **Space:** O(V)

P23 — Shortest Path in DAG *(GFG)*

Recall: Topo sort (DFS→stack) → then relax edges in topo order.

Algo: DFS topo sort. Pop from stack: `if(dist[node]!=1e9)` relax all neighbors.

Don't miss: - Only works on DAGs (no cycles) - Guard `dist[node]!=1e9` before relaxing — don't propagate infinity - Order matters: process in topo order, not arbitrary

Time: O(V+E) | **Space:** O(V)

P24 — Word Ladder I *(LC #127)*

Recall: BFS on implicit graph. Try all 26 char substitutions at each position. First reach = shortest.

Algo: `queue<{word,steps}>`. `unordered_set` as visited (erase on enqueue). Restore `word[i]` after each position.

Don't miss: - Erase from set on **enqueue** (not dequeue) - Restore `word[i]=original` after trying all 26 chars for position i - Erase `beginWord` from set first

Time: O(N×M×26) | **Space:** O(N×M)

Pattern: Implicit graph BFS (states + transitions) → BFS with set as visited

---

P25 — Word Ladder II *(LC #126)*

Recall: BFS storing full paths. Erase words at **level boundary**, not immediately.

Algo: `queue<vector<string>>`. `usedOnLevel` collects words used this level. Erase at `vec.size()>level`. Collect all same-length paths reaching endWord.

Don't miss: - `usedOnLevel` erased at level change, NOT immediately — multiple paths may use same word at same level - `vec.pop_back()` after pushing extended path (restore for next neighbor) - Collect only paths matching first answer's size

Time: O(N×M×26×L×paths) | **Space:** O(N×M×paths)

P26 — Dijkstra's Algorithm *(GFG/LC #743)*

Recall: Min-heap `{dist,node}`. Always process smallest distance. Relax neighbors.

PQ Algo: `dist[]=1e9`, `dist[src]=0`, push `{0,src}`. Pop → relax neighbors if `dis+w<dist[adj]`.

Set Algo: Same but use ordered set. Erase `{dist[adj],adj}` before inserting updated entry.

Don't miss: - Pair order: `{dist, node}` — heap sorts by FIRST element - `greater<pair<int,int>>` for min-heap (default is max-heap) - No negative weights (use Bellman-Ford for those)

Time: O((V+E)logV) | **Space:** O(V+E)

Pattern: Shortest path, non-negative weights → Dijkstra

P27 — Shortest Path + Path Reconstruction *(GFG)*

Recall: Dijkstra + `parent[]` array. Update `parent[adj]=node` on relaxation. Trace back from dest to src.

Don't miss: - `parent[i]=i` init (self-loop = source/unvisited) - `pq.push({0,src})` before while loop — easy to forget - Trace: `while(parent[node]!=node)` push, move. Then push src. Reverse.

Time: O((V+E)logV) | **Space:** O(V+E)

P28 — Shortest Distance in Binary Maze *(GFG + LC #1091)*

Recall: BFS on grid. Queue: `{dist,{r,c}}`. Unit cost per cell → BFS.

GFG: `1`=open, `0`=blocked. 4-directional. `dist[src]=0`. Return `dis+1` at dest. **LC #1091:** `0`=open, `1`=blocked. **8-directional**. `dist[src]=1` (counts start). Return `dis` at dest.

Don't miss: - GFG vs LC: inverted cell values, different direction counts, different dist init - Check blocked src/dest before BFS - `src==dest` → return 0 (GFG) or 1 (LC)

Time: O(n×m) | **Space:** O(n×m)

P29 — Path with Minimum Effort *(LC #1631)*

Recall: Modified Dijkstra. `dist[r][c]` = min possible max-diff to reach cell. `newEffort=max(diff,|h[r][c]-h[nr][nc]|)`.

Don't miss: - `max()` not `+` for effort accumulation — the ONE change from standard Dijkstra - Check dest on **pop** (not push) - `dist[0][0]=0`; return `diff` when dest popped

Time: O(n×m×log(n×m)) | **Space:** O(n×m)

Pattern: Minimize the MAXIMUM edge weight on path → Dijkstra with `max()` instead of `+`

P30 — Cheapest Flights K Stops *(LC #787)*

Recall: BFS (not Dijkstra) ordered by stop count. `queue<{stops,{node,cost}}>`.

Algo: `if(stops>k) continue`. Relax if `cost+w<dist[adj] && stops<=k`. Push `{stops+1,...}`.

Don't miss: - BFS not Dijkstra — stops increment by 1 each level → BFS ordering is valid - `dist[]` is pruning (not visited marker) — don't skip nodes already seen - Don't early-return at destination — cheaper path may come later - Directed graph

Time: O(V+E×K) | **Space:** O(V+E)

Pattern: Shortest path with hop constraint → BFS by constraint level

---

P31 — Network Delay Time *(LC #743)*

Recall: Dijkstra from source k. Answer = `max(dist[1..n])`. Any `1e9` → `-1`.

Don't miss: - Directed graph → one-way edges only - Stale skip: `if(dis>dist[node]) continue` - 1-indexed nodes

Time: O((V+E)logV) | **Space:** O(V+E)

Pattern: "Time for signal to reach ALL nodes" → Dijkstra + max of all distances

---

P32 — Minimum Multiplications to Reach End *(GFG)*

Recall: BFS on state space mod 1000. State = `(1LL*it*node)%1000`.

Don't miss: - `dist[1000]` indexed by mod value, not input size - `1LL*it*node` before mod — prevents int overflow - `start==end` → return 0 before BFS

Time: O(1000×|arr|) | **Space:** O(1000)

Pattern: Bounded state space + unit transitions → BFS on states

---

P33 — Number of Ways to Arrive at Destination *(LC #1976)*

Recall: Dijkstra + `ways[]` counting. Two relaxation cases.

Two cases: - `dis+w < dist[adj]` → update dist, `ways[adj]=ways[node]`, push - `dis+w == dist[adj]` → `ways[adj]=(ways[adj]+ways[node])%mod` (no push)

Don't miss: - Shorter → **reset** ways (not +=). Equal → **accumulate** ways. - `dist[]` must be `long long` (1e18 init) - Mod only on `ways[]`, never on `dist[]`

Time: O((V+E)logV) | **Space:** O(V+E)

Pattern: Count shortest paths → Dijkstra + ways[]. Shorter=reset, Equal=accumulate.

P34 — Bellman-Ford *(GFG)*

Recall: Relax ALL edges V-1 times. Nth pass: any relaxation → negative cycle.

Algo: `dist[V]=1e8, dist[src]=0`. V-1 passes over all edges: `if(dist[u]!=1e8 && dist[u]+wt<dist[v])`. Nth pass same check → return `{-1}`.

Don't miss: - `1e8` not `INT_MAX` — avoids overflow with negative weights - Guard `dist[u]!=1e8` — don't propagate unreachable - Nth pass = ONE extra pass (not V-1 again) - Works with negative weights; detects negative cycles

Time: O(V×E) | **Space:** O(V)

Pattern: Negative weights or cycle detection → Bellman-Ford

P35 — Floyd-Warshall *(GFG)*

Recall: All-pairs shortest path. Triple loop k(outer),i,j. `dist[i][j]=min(dist[i][j],dist[i][k]+dist[k][j])`.

Algo: Pre-process: `-1→1e9`, `diag=0`. Triple loop. Post-process: `1e9→-1`.

Negative cycle check: After algorithm: `matrix[i][i]<0` → cycle through node i.

Don't miss: - **k must be outermost** — DP dependency order (critical) - Pre AND post processing required - Overflow: `1e9+1e9=2e9` — within int range for this problem

Time: $O(V^3)$ | **Space:** $O(V^2)$ in-place

Pattern: All-pairs shortest path → Floyd-Warshall

P36 — Find City with Smallest Neighbours *(LC #1334)*

Recall: Floyd-Warshall → count cities within threshold per city → return min-count city (largest index on tie).

Don't miss: - `if(cnt<=cntCity)` with `<=` (not `<`) → ties go to **highest index** city - `cntCity=n` init (worst case) - Same Floyd-Warshall rules: k outer, pre/post-process

Time: $O(V^3)$ | **Space:** $O(V^2)$

P37 — MST Theory

Spanning Tree: V nodes, V-1 edges, connected, acyclic. **MST:** Spanning tree with minimum total weight.

Cut property: Min edge crossing any cut → in some MST (why Prim's works). **Cycle property:** Max edge in any cycle → never in MST (why Kruskal's works).

Prim's vs Kruskal's: | | Prim's | Kruskal's | |--|--------|----------| | Start | Single node | All edges | | Structure | Min-heap | Sort + DSU | | Better for | Dense | Sparse | | Cycle check | `vis[]` | DSU `findUPar` |

P38 — Prim's Algorithm *(GFG)*

Recall: Dijkstra-like but push raw **edge weight** (not cumulative). `vis[]` instead of `dist[]`.

Algo: `pq.push({0,0})`. Pop `{wt,node}`: if `vis[node]` skip. `vis[node]=1`, `sum+=wt`. Push unvisited neighbors with `edgeWeight`.

Don't miss: - Push `edgeWeight` not `dis+edgeWeight` (the ONE difference from Dijkstra) - Add weight on **pop** (confirmed in MST), not push - For MST edges: store `{wt,{adjNode,node}}` in heap, record `{parent,node}` on pop

Time: O((V+E)logV) | **Space:** O(V+E)

Pattern: MST on dense graph / Dijkstra-like → Prim's

P39 — Disjoint Set Union *(Theory)*

Two arrays: `parent[i]=i`, `size[i]=1`.

findUPar (path compression):

```
if node==parent[node]: return node
return parent[node]=findUPar(parent[node])
```

unionBySize: Find roots. Smaller size → child. `size[winner]+=size[loser]`.

Cycle check: `findUPar(u)==findUPar(v)` before union → same component.

Root count: `parent[i]==i` (O(1), after all unions). Not `findUPar(i)==i` (O(α(N))).

Don't miss: - Always union **roots**, not the nodes themselves - `ulp_u==ulp_v` → return early (same component) - 0-indexed: `resize(n)`. 1-indexed: `resize(n+1)`

Time: O(α(N)) per operation | **Space:** O(N)

P40 — Kruskal's Algorithm *(GFG)*

Recall: Sort edges by weight. For each edge: different roots → add to MST + union. Same root → skip (cycle).

Algo:

```
sort edges by edge[2] (weight)
for each (u,v,wt):
    if findUPar(u)!=findUPar(v): mstWt+=wt; union(u,v)
```

Don't miss: - Sort by `edge[2]` (weight), not node index - `findUPar` check BEFORE union - Each accepted edge = MST edge (collect directly for printing)

Time: O(E log E) | **Space:** O(V)

Pattern: MST on sparse graph / need MST edges directly → Kruskal's

P41 — Network Connected *(LC #1319)*

Recall: Min cable moves = `components-1`. Count extra (redundant) edges with DSU.

Algo: If `edges<n-1` → `-1`. DSU: same root → `extra++`; else union. Count roots. Return `components-1`.

Don't miss: - Early check `connections.size()<n-1` → only real guard needed - `extra++` when roots ARE equal (cycle edge = redundant cable) - `parent[i]==i` for root count (O(1)) - Final `-1` branch is dead code if early check passes

Time: O(E×α(N)) | **Space:** O(N)

Pattern: "Minimum moves to connect components" → components-1. DSU for counting.

P42 — Accounts Merge *(LC #721)*

Recall: DSU on account indices (not emails). Email triggers union between accounts.

Algo:

```
Phase 1: for each email in account i:
    not in map → mapMailNode[email]=i
    in map     → union(i, mapMailNode[email])

Phase 2: for each (email,idx) in map:
    mergedMail[findUPar(idx)].push_back(email)

Phase 3: for non-empty mergedMail[i]:
    sort emails. prepend accounts[i][0]. push to ans.
```

Don't miss: - DSU nodes = account **indices**, not emails - `j=1` in inner loop — skip the name at `accounts[i][0]` - Name: use `accounts[i][0]` where `i` is the root (mergedMail[i] non-empty ↔ i is root) - Sort emails within each account AND optionally sort final ans

Time: O(N×M×α(N) + N×M×log M) | **Space:** O(N×M)

Pattern: "Merge groups sharing a common element" → DSU with element as union trigger

---

Algorithm Selection Guide

```
Graph type / problem                    → Algorithm
──────────────────────────────────────────────────
Unweighted, unit cost                   → BFS
Any weights, non-negative, single src   → Dijkstra
Negative weights / detect neg cycle     → Bellman-Ford
All-pairs shortest path                 → Floyd-Warshall
DAG shortest path                       → Topo sort + relax
Shortest path + path reconstruction     → Dijkstra + parent[]
Minimize MAX edge weight on path        → Dijkstra with max()
Count shortest paths                    → Dijkstra + ways[]
K-stop constrained shortest path        → BFS by stop count
Connected components                    → DFS/BFS outer loop or DSU
Cycle - undirected                      → BFS/DFS + parent tracking
Cycle - directed                        → DFS + vis + pathVis
Topological order                       → DFS+stack or Kahn's BFS
Bipartite check                         → BFS 2-coloring
MST - dense graph                       → Prim's (min-heap)
MST - sparse graph / need edges         → Kruskal's (sort + DSU)
Dynamic connectivity / merge groups     → DSU
```

DSU Cheatsheet

```
class DSU {
public:
    vector<int> parent, size;
    DSU(int n) {
        parent.resize(n); size.resize(n, 1);
        for(int i=0;i<n;i++) parent[i]=i;
    }
    int findUPar(int node) {
        if(node==parent[node]) return node;
        return parent[node]=findUPar(parent[node]);
    }
    void unionBySize(int u, int v) {
        int pu=findUPar(u), pv=findUPar(v);
        if(pu==pv) return;
        if(size[pu]<size[pv]) { parent[pu]=pv; size[pv]+=size[pu]; }
        else                  { parent[pv]=pu; size[pu]+=size[pv]; }
    }
};
// Count components: parent[i]==i (O(1)) — NOT findUPar(i)==i
// Cycle check: findUPar(u)==findUPar(v) before union
// 1-indexed: resize(n+1), loop i=0..n
```

Complexity Summary

Algorithm	Time	Space	Notes
BFS / DFS	O(V+E)	O(V)	
Dijkstra	O((V+E)logV)	O(V+E)	Non-negative weights
Bellman-Ford	O(VE)	O(V)	Negative weights
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All-pairs
Topo sort	O(V+E)	O(V)	DAG only
Prim's MST	O((V+E)logV)	O(V+E)	
Kruskal's MST	O(E logE)	O(V)	Sort dominates
DSU (per op)	O(α(N))	O(N)	≈ O(1)
BFS on grid	O(n×m)	O(n×m)	
Dijkstra on grid	O(nm log nm)	O(nm)	